



Introduction

Subject, project title and project team

SUBJECT

SOFTWARE ENGINEERING AND PROJECT MANAGEMENT

CLASS PROJECT

PROJECT TITLE

Smart Scientific Calculator

A full-stack web application engineered with the Incremental Software Development Model.

Project Team

ROLE	NAME	ENROLMENT NO.
LEADER	SIDDHARTHA TIWARI	20243275
MEMBER	SOURABH VERMA	20243279
MEMBER	SINGH KRISHNARAJ	20243278
MEMBER	UMAR RASHEED	20243591

Smart Scientific Calculator

A full-stack web application engineered with the Incremental Software Development Model

AC	DEL	%	÷	√	x^y	sin	cos
7	8	9	×	1	2	3	+
4	5	6	-	0	.	=	DB

SUBMITTED BY	ENROLMENT / ROLL NO.
PROGRAMME & SEMESTER	INSTITUTION
COURSE INSTRUCTOR	DATE OF SUBMISSION October 2026

Executive Summary & Project at a Glance

Overview of the project, the report and the headline numbers

CONTENTS

1	Introduction	3
2	Software Process Model	4
3	Requirements Analysis	5
4	System Design	6
5	Implementation	7
6	Testing & Quality Assurance	8
7	Project Management	9
8	Conclusion & Future Scope	10
	References & Appendix	10

Executive summary

The **Smart Scientific Calculator** is a full-stack web application that performs arithmetic and scientific calculations, protects the user from invalid operations, and keeps a permanent record of every calculation in a database.

It was planned and delivered with the **Incremental Software Development Model**: six user needs (UN-01 to UN-06) were grouped into **five increments**, and each increment produced a working, testable version that added one clear capability to the one before it.

This report documents the whole life cycle – problem definition, process-model selection, requirements, design, implementation, testing and project management (scheduling, effort estimation, risk and configuration management) – and closes with an honest technical review and a roadmap for future work.

5

increments

6

user needs

20

features

329

lines of code*

3

architecture tiers

2

REST endpoints

*non-blank lines in server.js, index.html, script.js and style.css (comments included).

Project facts

Project title	Smart Scientific Calculator
Repository	github.com/Krishnarajss/Smart-Scientific-Calculator
Subject	Software Engineering and Project Management
Process model	Incremental Software Development Model – 5 increments covering 6 user needs (UN-01 ... UN-06)
Client side	HTML5 keypad, CSS3 grid styling, JavaScript ES6 classes (CalculatorUI, InputHandler, CalculationEngine)
Server & data	Node.js + Express 4 REST API · Mongoose 8 · MongoDB (default URI mongodb://127.0.0.1:27017/calculator_db)
Run	npm start → node server.js → http://localhost:3000

Code at a glance

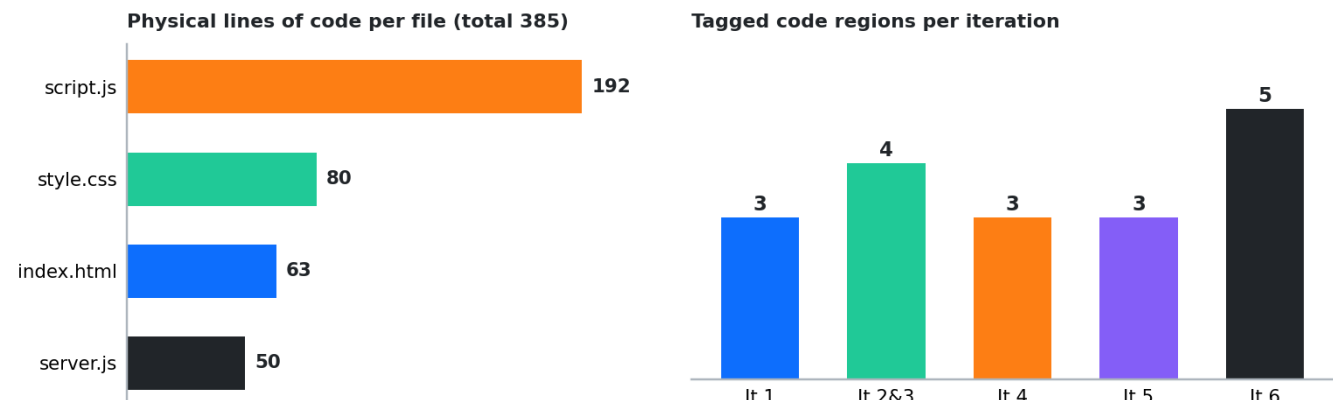


Figure 1: lines of code per file (left) · Figure 2: iteration-tagged code regions – the shared CSS block is counted for both Iteration 1 and 5 (right)

1

Introduction

Background, problem, objectives, scope and technology

1.1 Background

Calculators are among the most familiar software tools, yet building one well exercises almost every discipline of software engineering: requirements, modularity, input validation, error handling, interface design and persistence. Ordinary calculators solve arithmetic but forget what the user did, and many fail ungracefully on invalid input such as division by zero. The Smart Scientific Calculator closes these gaps and doubles as a vehicle for demonstrating a disciplined software process.

1.2 Problem statement

Users need a quick, reliable, browser-based calculator that performs basic *and* scientific operations, reports mistakes clearly instead of crashing or printing meaningless values such as *Infinity* or *NaN*, is comfortable to use, and remembers past calculations. Delivering all of this in one big release is risky for a small team, so a staged, low-risk approach is required.

Problem observed		Solution delivered
Calculators forget your work	→	Every result is saved in MongoDB and can be recalled (UN-06)
Invalid input crashes or shows Infinity / NaN	→	Errors are caught and shown as a clean "Error" (UN-04)
Basic tools lack scientific functions	→	Square root, power, sine and cosine are provided (UN-02, UN-03)
Layouts are cluttered and hard to read	→	Colour-coded 4-column grid with a live display (UN-05)

1.3 Objectives

O1	Deliver a working, tested product after every iteration.
O2	Separate calculation logic, input control, user interface and persistence into distinct layers.
O3	Handle invalid input gracefully so the application never crashes.
O4	Store and retrieve calculation history permanently with MongoDB.
O5	Apply and document SEPM practice: process model, traceability, estimation, risk and configuration management.

1.4 Scope and stakeholders

In scope (current version)	Out of scope (not implemented)
+ − × ÷ and % keys; AC and DEL x^y , $\sqrt{}$, sin, cos (angles in degrees) Error handling for ÷0, invalid domains and syntax errors Grid keypad with live expression display REST API and MongoDB history (latest 10 records)	User accounts and authentication tan, log, ln, factorial, parentheses, radian mode Keyboard input and memory keys Automated unit tests and CI pipeline Cloud deployment of the application

Stakeholders: the end user of the calculator, the developer(s) who build and maintain it, and the course instructor who evaluates the project.

1.5 Technology stack

Layer	Technology	Version	Role in the project
Markup	HTML5	-	Keypad buttons and display structure (index.html)
Styling	CSS3	-	Grid layout and colour-coded keys (style.css)
Client logic	JavaScript ES6	-	CalculationEngine, InputHandler, CalculatorUI classes (script.js)
Runtime	Node.js	-	Runs the server with npm start
Web framework	Express	^4.19.2	Serves the UI and exposes the REST API
Middleware	cors	^2.8.5	Allows cross-origin requests
ODM	Mongoose	^8.3.1	Schema and model for the History collection
Database	MongoDB	-	Permanent storage of calculation history
Version control	Git + GitHub	-	Source hosting and change history

2

Software Process Model

Why the Incremental Development Model was chosen and how it was applied

2.1 The incremental model

In the incremental model the product is built in a series of small, usable pieces. Each increment passes through its own mini life-cycle – analysis, design, coding and testing – and is added to the software delivered so far, so a working product exists from the very first iteration. Functionality grows steadily while risk is spread over time [1][5].

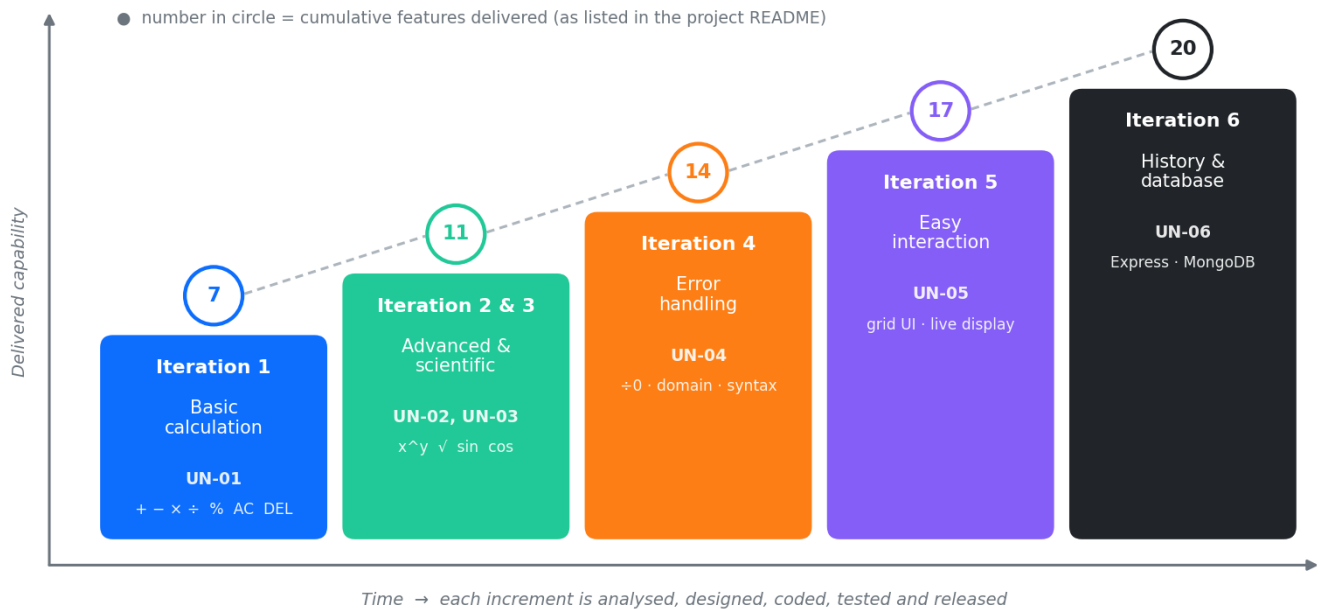


Figure 3: the five increments of the project. Each step keeps everything delivered before and adds one new capability.

2.2 Every increment repeats the same mini life-cycle



2.3 Why incremental fits this project

- ▶ **Clear, modular requirements.** The six user needs are largely independent, so they map naturally to separate increments.
- ▶ **Early working software.** A usable calculator exists after Iteration 1; later increments only enrich it.
- ▶ **Risk control.** The riskiest piece – database integration – is deferred to Iteration 6, after the core logic is stable.
- ▶ **Easy testing.** Each increment is tested on its own and the earlier increments are re-tested (regression) to protect what already works.
- ▶ **Fits a small team and fixed deadline.** If time runs short, a complete earlier increment can still be submitted.

2.4 Process-model selection matrix

Criterion	Weight	Waterfall	Incremental	Spiral	Agile (Scrum)
Requirements are clear and stable	20%	5	4	3	3
Working software available early	25%	1	5	3	5
Handles technical risk	20%	2	4	5	4
Flexible to change	15%	1	4	4	5
Low management overhead	20%	4	4	2	3
Weighted score (out of 5)	100%	2.60	4.25	3.35	4.00

Qualitative assessment by the project author (1 = poor, 5 = excellent); weights reflect this project's priorities. Incremental scores highest (4.25), ahead of Agile (4.00), Spiral (3.35) and Waterfall (2.60).

3

Requirements Analysis

User needs, traceability, functional and non-functional requirements

3.1 User needs and requirements traceability matrix

The README defines six user needs. The matrix below links each one to the iteration that delivered it, the code that implements it and the test cases (Section 6) that verify it – the basis of requirements traceability.

ID	User need	Iter.	Implemented in	Verified by
UN-01	Basic calculation: + – × ÷, %, clear (AC), delete (DEL)	1	Keypad in index.html; InputHandler; expression evaluation in displayResult()	T01-T04, T13, T14
UN-02	Advanced calculation: power x^y	2 & 3	applyScientific('pow'); CalculationEngine.power()	T10
UN-03	Scientific functions: $\sqrt{}$, sin, cos	2 & 3	CalculationEngine.squareRoot(), sin(), cos()	T08, T11, T12
UN-04	Error handling without crashing	4	Zero-divide guard, isFinite() check, try / catch blocks	T05-T07, T09
UN-05	Easy interaction: clear layout, live display, touch feedback	5	CSS grid, updateDisplay(), .btn:active	Visual / usability check
UN-06	Calculation history and database	6	server.js routes, History model, toggleHistory()	T16, T17

3.2 Functional requirements

FR1 Accept digits, the decimal point and operators from an on-screen keypad.

FR2 Evaluate expressions using + – × ÷ and %.

FR3 Compute x^y (exponent requested from the user), $\sqrt{}$, sin and cos (degrees).

FR4 Clear everything (AC) and delete the last character (DEL).

FR5 Show the running expression and the last “expression = result” line.

FR6 Detect division by zero, invalid root domain, syntax errors and non-finite results and show “Error”.

FR7 Save every successful calculation (expression, result, time) through POST /api/history.

FR8 Show the 10 most recent saved calculations on demand (GET /api/history).

3.3 Non-functional requirements

Usability colour-coded 4-column keypad with large touch targets and press feedback.

Reliability invalid input never crashes the interface.

Modularity separate classes for calculation, input and UI.

Maintainability every code region is tagged // Iteration n: start ... end for traceability.

Portability runs in any modern browser; server runs on Node.js.

Performance calculations are instant; history is one small JSON request.

Configurability database address supplied through the MONGO_URI variable.

Persistence history survives page reloads and server restarts.

3.4 Use-case model

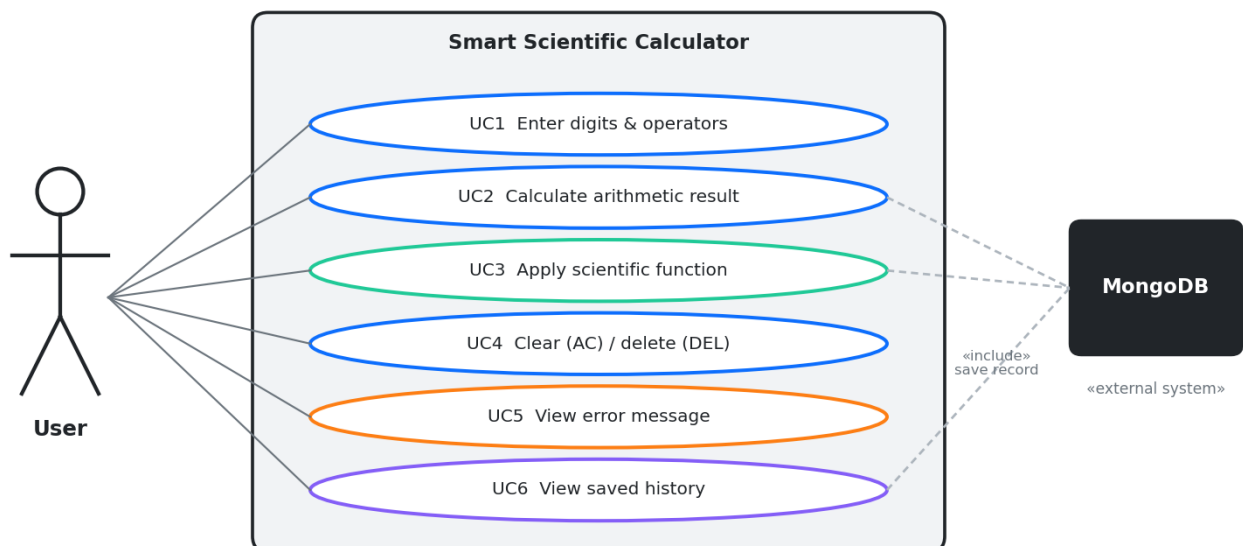


Figure 4: use-case diagram – one primary actor (User) and one external system (MongoDB).

4

System Design

Three-tier architecture, class design, data model and REST API

4.1 Architecture

The system follows a classic **three-tier** structure. The browser hosts the interface and the calculation logic; an Express server serves the static files and exposes a small REST API; MongoDB, accessed through Mongoose, stores the history. Separating the tiers lets each one change independently.

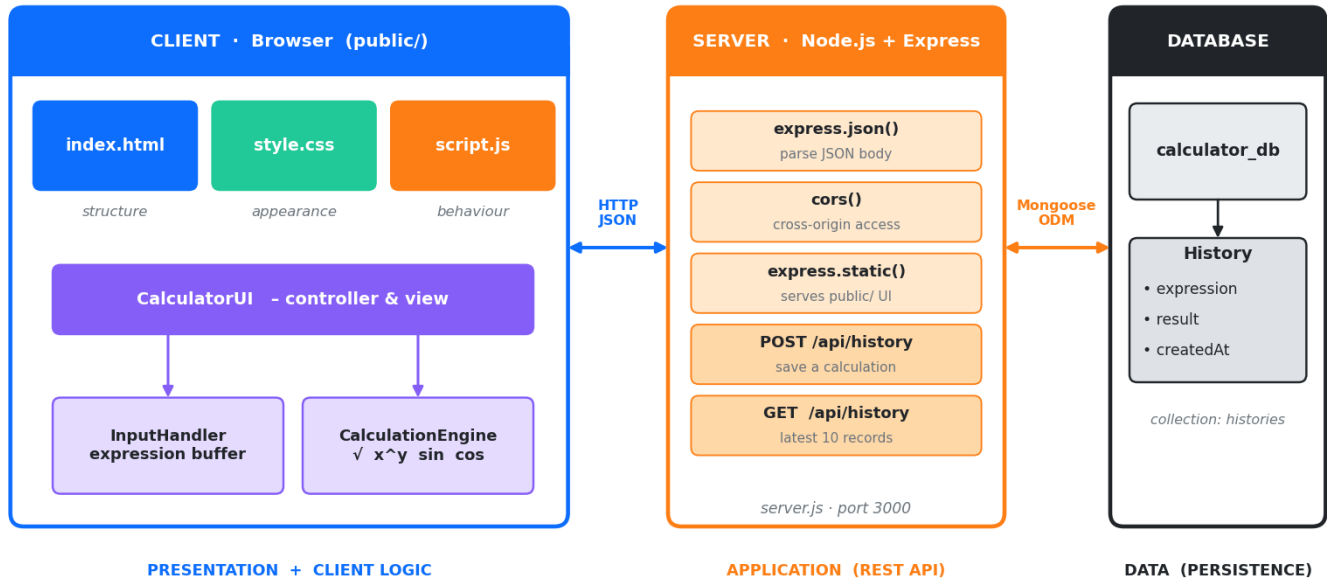


Figure 5: architecture of the Smart Scientific Calculator.

4.2 Class design

CalculatorUI is the controller and view: it owns an **InputHandler** (the expression buffer) and a **CalculationEngine** (mathematical rules such as the zero-divide and negative-root checks). This applies the single-responsibility and separation-of-concerns principles.

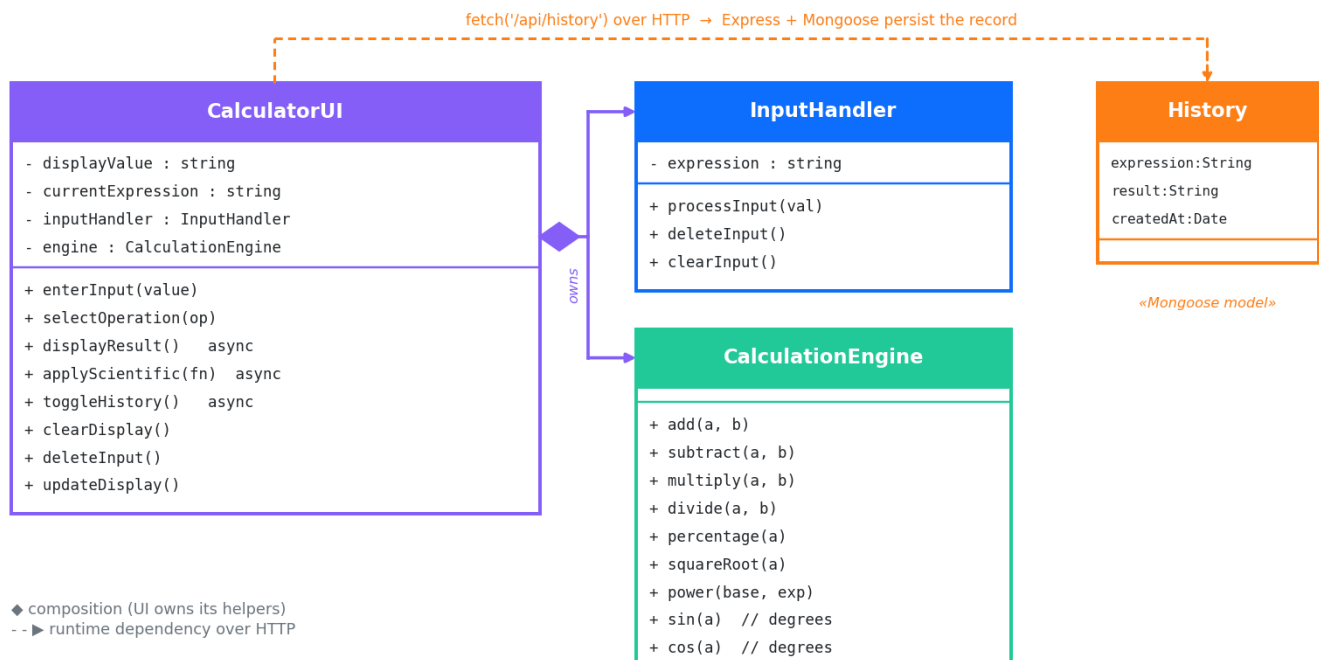


Figure 6: class diagram of the client code and the History model.

4.3 Data model

History field	Type	Rule
expression	String	required
result	String	required
createdAt	Date	default: Date.now

4.4 REST API

Method & path	Request body	Success / failure
POST /api/history	{ expression, result }	201 + saved record / 500 { error }
GET /api/history	-	200 + newest 10 records / 500 { error }

5

Implementation

How each increment was built, the user interface and the main runtime flow

5.1 Iteration-by-iteration implementation

Iter.	What was built	Key code
1	Keypad buttons call methods of <code>calculatorUI</code> ; <code>InputHandler</code> builds the expression string; “=” evaluates it.	<code>this.inputHandler.processInput(val);</code>
2 & 3	<code>applyScientific()</code> reads the current value, calls the engine and stores the result as the new expression; x^y asks for the exponent.	<code>sin(a) { return Math.sin(a * Math.PI / 180); }</code>
4	Domain guards in the engine, an <code>isFinite()</code> check and try / catch turn every failure into a clean “Error”.	<code>if (b === 0) throw new Error("Division by zero");</code>
5	Dark card, live display, colour-coded buttons in a 4-column grid, press animation.	<code>.keypad { display: grid; grid-template-columns: repeat(4, 1fr); }</code>
6	Express routes and a Mongoose model; the UI POSTs each result and lists the last 10 saved calculations.	<code>History.find().sort({ createdAt: -1 }).limit(10)</code>

Traceability by design. Every region of code carries a comment pair such as `// Iteration 4: start4 ... end4`, so any line can be linked back to the user need it satisfies.

■ Function keys – AC, DEL, history

■ Operators – % ÷ × − +

■ Number keys

■ Equals

■ Scientific – √ x^y sin cos



Figure 7: the calculator interface, drawn from `index.html` and `style.css`.

5.2 Runtime flow of a calculation

When the user presses “=” the expression is evaluated, validated with `isFinite()`, sent to the REST API for storage, and only then shown as the new result. Any exception – for example a syntax error – is caught and the display shows “Error”.

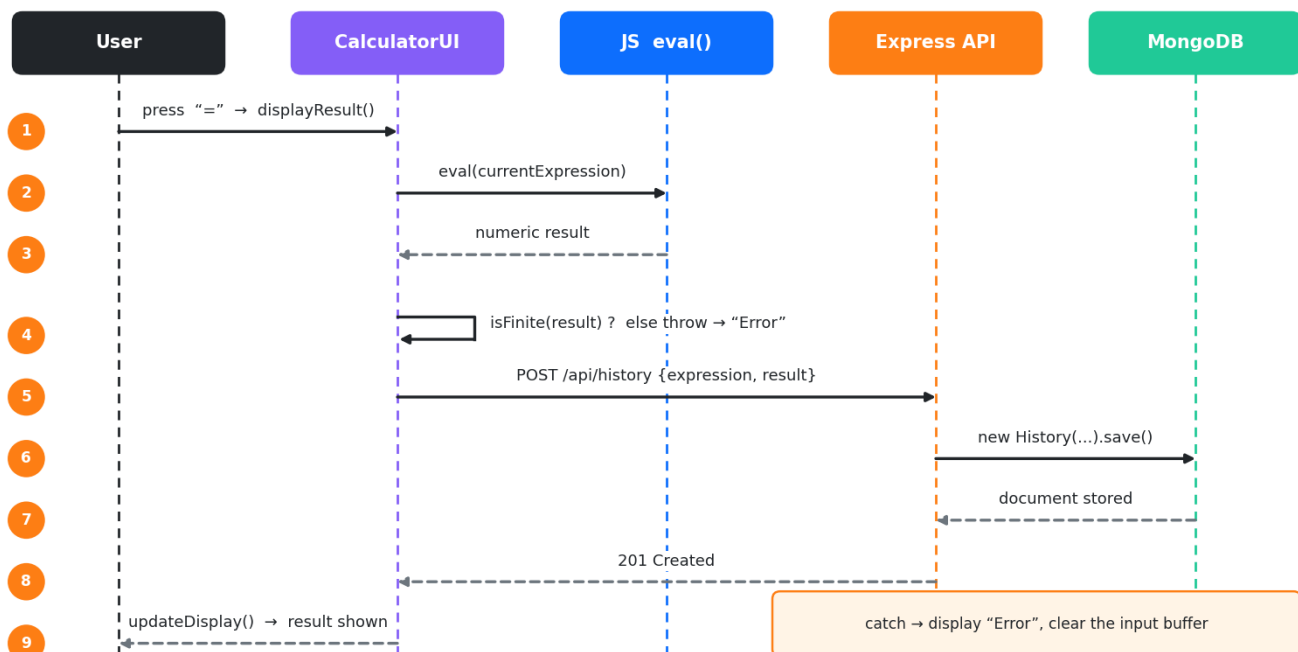


Figure 8: sequence diagram of one successful calculation (steps 1–9) and the error path.

6

Testing & Quality Assurance

Black-box test cases, verified results and a technical review of the finished code

6.1 Test cases and results

Test cases T01-T15 were **executed** with Node.js using the project's own *CalculationEngine* class (loaded from script.js) and the same *eval()* + *isFinite()* rules as the UI, covering normal, boundary and invalid inputs (equivalence partitioning and boundary-value analysis). T16-T17 need a live MongoDB instance and were checked by code inspection.

ID	Test case (input)	Expected	Observed	Result
T01	2 + 3	5	5	PASS
T02	7 × 8	56	56	PASS
T03	10 − 4	6	6	PASS
T04	9 ÷ 3	3	3	PASS
T05	5 ÷ 0	Error	Error - Infinity caught by isFinite()	PASS
T06	0 ÷ 0	Error	Error - NaN caught by isFinite()	PASS
T07	12 + (incomplete)	Error	Error - SyntaxError caught	PASS
T08	√ 144	12	12	PASS
T09	√ (−9)	Error	Error - "Invalid domain for root"	PASS
T10	2 ^ 10	1024	1024	PASS
T11	sin 90°	1	1	PASS
T12	sin 30°	0.5	0.49999999999999994	NOTE
T13	2 + 3 × 4	14 (precedence)	14	PASS
T14	50 % 200	percent semantics (e.g. 0.5)	50 - % acts as JavaScript remainder	NOTE
T15	0.1 + 0.2	0.3	0.30000000000000004	NOTE
T16	POST /api/history	201 + saved record	Requires running MongoDB - verified by code review	REVIEW
T17	GET /api/history	10 newest, latest first	Requires running MongoDB - verified by code review	REVIEW

Summary: 12 pass, 3 notes (floating-point display, % semantics), 2 reviewed. No executed test produced a crash; all invalid inputs were handled with "Error".

6.2 Technical review: findings and recommendations

#	Finding	Evidence in code	Severity	Recommended fix
F1	% key behaves as remainder, not percentage	% is inserted into the string evaluated by eval(); engine.percentage() is never called	Medium	Call percentage() or define percent-of semantics
F2	Arithmetic bypasses the engine and relies on eval()	displayResult() - only √, x^y, sin, cos use the engine	Medium	Use a safe expression parser and the engine methods
F3	Scientific keys use only the first number	parseFloat("2 + 3") is 2, so sin acts on 2	Medium	Evaluate the expression first, then apply the function
F4	Display waits for the network call	await fetch() precedes updateDisplay(); a failed request shows "Error" for a valid result	Medium	Show the result first; save history asynchronously
F5	node_modules is committed; no .gitignore	Repository root	Low	Add .gitignore; install with npm install
F6	Floating-point noise is visible	String(result) is shown unrounded	Low	Round to about 12 significant digits for display
F7	Open CORS, fixed port, no payload validation	cors(); PORT = 3000 in server.js	Low	Restrict origins; read PORT from environment
F8	No automated tests	No test script in package.json	Low	Add Jest tests and a GitHub Actions workflow

Reading the findings: these items are normal *technical debt* for an incrementally built student project - none stops the calculator from working for everyday use. Listing them honestly, with a fix for each, is part of good software quality management.

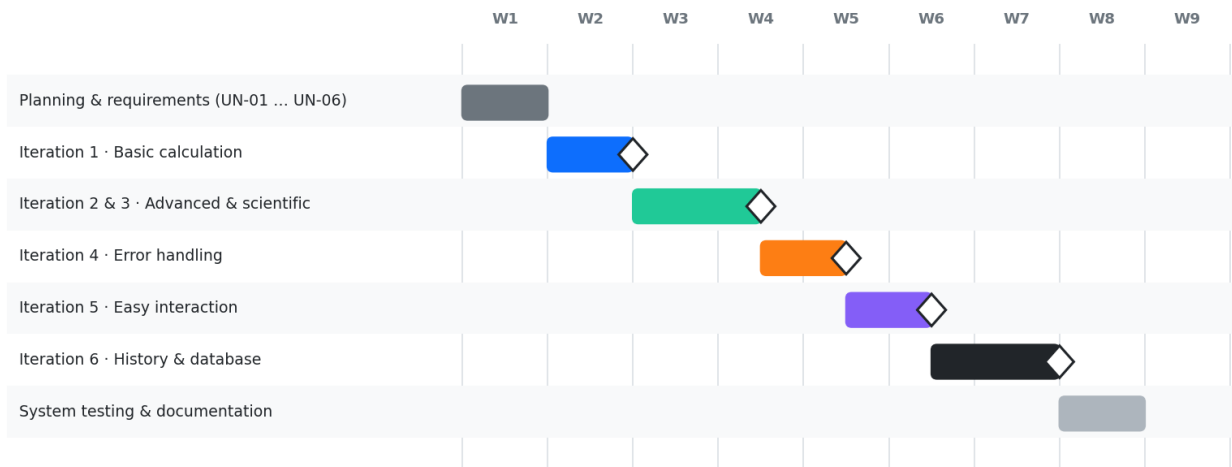
7

Project Management

Schedule, effort estimation, risk management and configuration management

7.1 Work breakdown and schedule

The work breakdown has four branches – planning, five iterations, system testing and documentation – shown as a Gantt chart with one milestone per increment.



◇ M1-M5 = release of a working increment. Illustrative planned schedule (weeks), not recorded actuals.

Figure 9: illustrative iteration schedule (planned, in weeks).

7.2 Effort estimation

Basic COCOMO - organic mode	Value
Size (non-blank lines ÷ 1000)	0.329 KLOC
Effort $E = 2.4 \times \text{KLOC}^{1.05}$	0.75 person-months
Time $D = 2.5 \times E^{0.38}$	2.24 months
Average staff $E \div D$	0.33 persons
Productivity $\text{SLOC} \div E$	440 lines / person-month

COCOMO [4] is calibrated for much larger systems, so for a project of this size it is only an order-of-magnitude check.

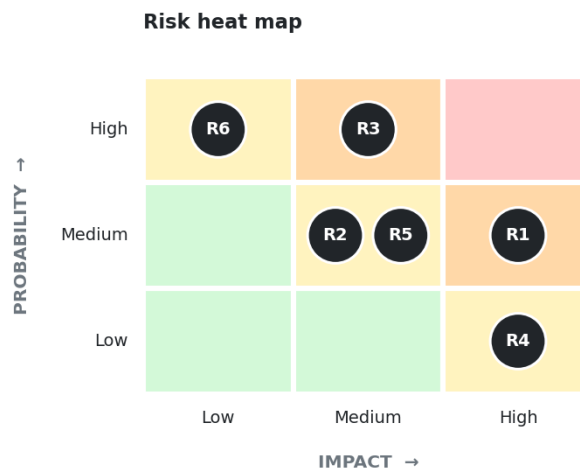


Figure 10: risk probability / impact matrix (R1-R6).

7.3 Risk management

ID	Risk	P	I	Mitigation	Status
R1	Database unavailable or slow	M	H	MONGO_URI is configurable; errors are logged; history failure shows an alert (see F4)	Open
R2	Invalid input crashes the UI	M	M	try / catch and isFinite() guards; domain checks in the engine (Iteration 4)	Mitigated
R3	Scope creep between increments	H	M	Scope fixed per increment; new ideas parked for later iterations	Controlled
R4	Unsafe use of eval()	L	H	Input comes only from the fixed keypad; replace with a parser (F2)	Open
R5	Regression when an increment is added	M	M	Re-run earlier test cases T01-T15 after every increment	Controlled
R6	Floating-point results confuse users	H	L	Round displayed values (F6)	Open

7.4 Configuration and quality management

- ▶ **Version control and baselines:** Git and GitHub hold the source and the README documents the iteration plan; the // Iteration n: start ... end tags act as lightweight baselines.
- ▶ **Dependencies:** package.json and package-lock.json give reproducible installs; node_modules should be excluded with a .gitignore (F5).
- ▶ **Quality assurance:** test cases T01-T17, the traceability matrix (Section 3.1) and the review findings F1-F8 form the QA record.

8

Conclusion & Future Scope

What was achieved, what comes next and how to run the project

8.1 Conclusion

The Smart Scientific Calculator shows how even a small product benefits from a disciplined process. Five increments delivered six user needs; each one is traceable to a code region, a test case and a schedule milestone. The architecture separates presentation, logic and data, and the project was supported by estimation, scheduling, risk and configuration management. The technical review also shows where debt remains – chiefly eval()-based arithmetic, the % semantics and the lack of automated tests – and each item has a concrete remedy.

8.2 Objectives versus outcomes

Objective	Outcome	Evidence	Status
01 Working product after every iteration	Five releasable increments, milestones M1-M5	Section 2, Figure 3, Figure 9	ACHIEVED
02 Separate logic, input, UI and persistence	Three tiers and three client classes; arithmetic still uses eval()	Figures 5-6, finding F2	LARGELY
03 Graceful handling of invalid input	÷0, 0÷0, syntax errors and negative roots all show "Error"	Tests T05-T07, T09	ACHIEVED
04 Permanent history in MongoDB	POST and GET endpoints with a Mongoose model	Section 4.3-4.4, T16-T17	ACHIEVED
05 Apply SEPM practice	Model selection, traceability, COCOMO, risk register, configuration management	Sections 2, 3.1 and 7	ACHIEVED

8.3 Future scope

NOW · quick wins	NEXT · features & quality	LATER · scale & reach
▶ Replace eval() with a safe expression parser ▶ Use engine methods; fix % semantics ▶ Round displayed results ▶ Show the result before saving history ▶ Add .gitignore; remove node_modules	▶ tan, log, ln, factorial, π and parentheses ▶ Degree / radian switch ▶ Keyboard input ▶ History panel with clear option instead of alert() ▶ Jest unit tests + GitHub Actions CI	▶ User accounts and per-user history ▶ Docker image and cloud MongoDB ▶ Installable offline PWA ▶ Accessibility audit (contrast, ARIA) ▶ Responsive layout with media queries

8.4 Lessons learned

- ▶ Delivering in increments gave a testable product early and made progress visible.
- ▶ Code tags linked to user needs keep requirements and implementation connected.
- ▶ Persistence should never block the core function – decoupling it (F4) is the next design improvement.
- ▶ Automated tests written early would have caught the % behaviour (F1) in Iteration 1.

Appendix A - Running the project

```
git clone https://github.com/Krishnarajss/Smart-Scientific-Calculator.git
cd Smart-Scientific-Calculator
npm install # express, mongoose, cors
# start MongoDB locally, or set MONGO_URI to your own connection string
npm start # opens the app at http://localhost:3000
curl http://localhost:3000/api/history # check the REST API
```

References

- [1] I. Sommerville, *Software Engineering*, 10th ed., Pearson, 2016.
- [2] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed., McGraw-Hill, 2019.
- [3] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 7th ed., 2021.
- [4] B. W. Boehm, *Software Engineering Economics*, Prentice-Hall, 1981.
- [5] C. Larman and V. R. Basili, "Iterative and Incremental Development: A Brief History," *IEEE Computer*, vol. 36, no. 6, 2003.
- [6] Documentation: Express (expressjs.com), Mongoose (mongoosejs.com), MongoDB (mongodb.com/docs), MDN Web Docs (developer.mozilla.org).
- [7] Krishnarajss, *Smart-Scientific-Calculator*, source code and README, github.com/Krishnarajss/Smart-Scientific-Calculator.